

Implementación manual de K-Means sobre datos de tracking de jugadores de la NFL

Momento de Retroalimentación: Módulo 2 — Implementación de una técnica de aprendizaje máquina sin el uso de un framework (Portafolio Implementación)

Karol Alexis Alvarado Davila — A01751711

Inteligencia artificial avanzada para la ciencia de datos

Grupo TC3009C.601

Tecnológico de Monterrey

Miércoles 2 de septiembre de 2026

1. Descripción del dataset

El dataset que usé es de tracking de jugadores de la NFL (archivo `input_2023_w01.csv`), que corresponde a la semana 1 de la temporada 2023. Tiene 285,714 registros y 23 columnas, sin valores nulos. Cada registro es un instante (frame) de un jugador dentro de una jugada, así que un mismo jugador aparece muchas veces: en total hay 16 partidos, 819 jugadas y 737 jugadores distintos, con unos 28 frames por jugador en cada jugada.

Las columnas se pueden agrupar así:

- Identificadores: `game_id`, `play_id`, `nfl_id`, `frame_id`.
- Datos del jugador: `player_name`, `player_height`, `player_weight`, `player_birth_date`, `player_position`, `player_side`, `player_role`.
- Contexto de la jugada: `play_direction`, `absolute_yardline_number`, `player_to_predict`, `num_frames_output`, `ball_land_x`, `ball_land_y`.
- Movimiento del jugador en ese instante: `x`, `y` (posición en el campo, en yardas), `s` (velocidad, yd/s), `a` (aceleración, yd/s²), `dir` (dirección del movimiento, en grados) y `o` (orientación del cuerpo, en grados).

2. Variables utilizadas y justificación

Para el clustering usé solamente dos variables: la velocidad `s` y la aceleración `a`. En el Cuadro 1 está el resumen estadístico de las dos en el dataset completo.

Variable	Media	Desv. est.	Mín.	Mediana	Máx.
<code>s</code> (yd/s)	3.04	2.23	0	2.73	12.53
<code>a</code> (yd/s ²)	2.13	1.43	0	1.92	17.12

Cuadro 1: Estadísticas de las variables utilizadas (285,714 registros).

La razón es que K-Means agrupa por distancias, así que las variables tienen que ser numéricas y tener un significado que valga la pena comparar. Con eso en mente, descarté las demás:

- `game_id`, `play_id`, `nfl_id` y `frame_id` son identificadores. Aunque sean números, la distancia entre dos ids no significa nada.

- `x` y `y` indican *dónde* está el jugador en el campo, y eso depende de en qué yarda empieza la jugada y hacia qué lado se juega, no de cómo se mueve. Si las hubiera incluido, K-Means habría agrupado por zona del campo en lugar de por tipo de movimiento.
- `dir` y `o` son ángulos de 0 a 360 grados. Con la distancia euclidiana que vimos en clase, 359° y 1° quedan muy lejos aunque en realidad sean casi la misma dirección, así que no son apropiadas para este algoritmo tal cual.
- `player_weight` y `player_height` son características fijas del jugador, no del movimiento.
- `player_role`, `player_position`, `player_side` y `play_direction` son categorías. No las convertí a números porque asignarles un número arbitrario inventaría una distancia que no existe.

Entre `s` y `a` la correlación es de 0.21, o sea que no representan la misma información: un jugador puede ir rápido sin acelerar, o acelerar desde velocidad cero.

3. Objetivo del proyecto

El objetivo es agrupar los registros de tracking según su velocidad y aceleración para encontrar tipos de movimiento parecidos (por ejemplo: jugador detenido, jugador acelerando, jugador corriendo a velocidad alta), y con eso demostrar que mi implementación manual de K-Means funciona. Es un problema de clustering, no de clasificación: no hay una etiqueta correcta que predecir, el algoritmo tiene que descubrir los grupos por su cuenta.

4. K-Means

K-Means es un algoritmo de aprendizaje no supervisado, de tipo centroe: cada cluster está representado por un centroe y cada punto pertenece al cluster cuyo centroe le queda más cerca. Lo que trata de minimizar es la inercia, que es la suma de las distancias al cuadrado de cada punto a su centroe. Los pasos, tal como los vimos en clase, son:

1. Elegir el número de clusters K .
2. Tomar K puntos aleatorios del dataset como centroides iniciales.
3. Asignar cada punto al centroe más cercano.
4. Calcular los nuevos centroides como el promedio de los puntos de cada cluster.
5. Repetir los pasos 3 y 4 hasta cumplir un criterio de paro.

Como criterio de paro usé los dos que vimos: que los centroides ya no cambien o que se llegue a un máximo de iteraciones. Para la distancia usé la euclidiana, que para dos puntos $P_1 = (x_1, \dots, x_n)$ y $P_2 = (y_1, \dots, y_n)$ es

$$D(P_1, P_2) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}.$$

5. Implementación manual

Todo lo que es K-Means está programado desde cero en `proyecto.py`, con funciones sencillas, ciclos `for` y listas. No importé `sklearn.cluster` ni ninguna función que haga parte del algoritmo. Las funciones son:

- `distancia(p1, p2)`: distancia euclidiana entre dos puntos.
- `centroides_iniciales(datos, k)`: toma los primeros k puntos del conjunto de entrenamiento. Como `train_test_split` ya revolvió los datos, esos k puntos son puntos aleatorios del dataset, que es lo que pide el algoritmo.
- `asignar_clusters(datos, centroides)`: calcula la distancia de cada punto a todos los centroides y lo asigna al más cercano. Esta misma función es la que uso después para asignar los datos de prueba.
- `calcular_centroides(datos, asignaciones, centroides)`: saca el promedio de los puntos de cada cluster.
- `inercia(datos, asignaciones, centroides)`: suma de distancias al cuadrado de cada punto a su centroide.
- `kmeans(datos, k, max_iteraciones)`: junta todo en un ciclo `while` que se detiene cuando los centroides ya no cambian o se llega al máximo de iteraciones.
- `indice_dunn(datos, asignaciones, centroides)`: métrica de separación entre clusters (ver Sección 10).

Lo que no es K-Means lo hice con las herramientas que usamos en clase: `pandas` para leer el CSV y seleccionar columnas, `StandardScaler` de `sklearn` para estandarizar, `train_test_split` para separar entrenamiento y prueba, y `matplotlib` para las gráficas. El Cuadro 2 resume qué parte es manual y qué parte usa herramientas.

Parte del proyecto	Cómo se hizo
Leer el CSV, explorar y seleccionar <code>s</code> y <code>a</code>	<code>pandas</code> (visto en clase)
Tomar la muestra (1 de cada 50 registros)	slicing de <code>pandas</code>
Estandarizar las variables	<code>StandardScaler</code> (visto en clase)
Separar entrenamiento y prueba	<code>train_test_split</code> (visto en clase)
Centroides iniciales, distancias, asignación, actualización, iteraciones, paro	manual
Asignación de los datos de prueba a un cluster	manual
Inercia, método del codo e índice de Dunn	manual
Gráficas	<code>matplotlib</code> (visto en clase)

Cuadro 2: Qué se implementó manualmente y qué usa herramientas vistas en clase.

Estas son las funciones principales del algoritmo, tal como están en `proyecto.py`:

```
1 def distancia(p1, p2):
2     # distancia euclidiana
3     suma = 0
4     for i in range(len(p1)):
5         suma = suma + (p1[i] - p2[i]) ** 2
```

```

6         return suma ** 0.5
7
8     def asignar_clusters(datos, centroides):
9         # cada punto se va al centroide mas cercano
10        asignaciones = []
11        for punto in datos:
12            cluster_cercano = 0
13            distancia_minima = distancia(punto, centroides[0])
14            for j in range(1, len(centroides)):
15                d = distancia(punto, centroides[j])
16                if d < distancia_minima:
17                    distancia_minima = d
18                    cluster_cercano = j
19            asignaciones.append(cluster_cercano)
20        return asignaciones
21
22    def calcular_centroides(datos, asignaciones, centroides):
23        # el nuevo centroide es el promedio de los puntos del cluster
24        k = len(centroides)
25        dimensiones = len(datos[0])
26        nuevos_centroides = []
27        for j in range(k):
28            suma = [0] * dimensiones
29            cuenta = 0
30            for i in range(len(datos)):
31                if asignaciones[i] == j:
32                    for d in range(dimensiones):
33                        suma[d] = suma[d] + datos[i][d]
34                    cuenta = cuenta + 1
35            if cuenta > 0:
36                promedio = []
37                for d in range(dimensiones):
38                    promedio.append(suma[d] / cuenta)
39                nuevos_centroides.append(promedio)
40            else:
41                # cluster vacio, se queda el centroide anterior
42                nuevos_centroides.append(centroides[j])
43        return nuevos_centroides
44
45    def kmeans(datos, k, max_iteraciones):
46        centroides = centroides_iniciales(datos, k)
47        historial_inercia = []
48        iteracion = 0
49        while True:
50            asignaciones = asignar_clusters(datos, centroides)
51            nuevos_centroides = calcular_centroides(datos, asignaciones,
52                                                    centroides)
53            iteracion = iteracion + 1
54            historial_inercia.append(inercia(datos, asignaciones, centroides
55            ))
56            # paro cuando los centroides ya no cambian o se acaban las
57            iteraciones
58            if nuevos_centroides == centroides or iteracion ==
59            max_iteraciones:
60                break
61            centroides = nuevos_centroides
62        asignaciones = asignar_clusters(datos, centroides)
63        return centroides, asignaciones, historial_inercia

```

6. Datos de entrenamiento y datos de prueba

6.1. Muestra

El dataset original tiene 285,714 registros. Como K-Means está programado con ciclos `for`, correrlo con todos los registros (y además varias veces para el método del codo) sería muy lento sin que el resultado cambie en nada importante, porque lo que quiero es demostrar que el algoritmo funciona. Por eso tomé una muestra de 1 registro de cada 50 usando slicing (`df[:50]`), lo que da 5,715 registros repartidos entre los 16 partidos del archivo.

6.2. Escalamiento

Como K-Means usa distancias, estandaricé las dos variables con `StandardScaler`, igual que en clase (media 0 y desviación estándar 1). Así ninguna de las dos pesa más que la otra solo por tener valores más grandes.

6.3. Separación

Separé la muestra con `train_test_split` en 80 % para entrenamiento y 20 % para prueba, con `random_state=1` para que el resultado sea el mismo cada vez que se corre el programa (Cuadro 3).

Conjunto	Registros
Dataset original	285,714
Muestra (1 de cada 50)	5,715
Entrenamiento (80 %)	4,572
Prueba (20 %)	1,143

Cuadro 3: Datos utilizados para entrenar y para probar.

El conjunto de **entrenamiento** se usó para todo lo que es aprender: el método del codo y el cálculo de los centroides finales. El conjunto de **prueba** no se tocó durante el entrenamiento; solo se usó al final para comprobar cómo asigna el algoritmo datos nuevos a los clusters y para calcular las métricas sobre datos que no había visto.

7. Número de clusters

Para escoger K usé el método del codo que vimos en clase: corrí mi K-Means en el conjunto de entrenamiento con K de 1 a 8 y calculé la inercia (WCSS) de cada resultado, con mi propia función (Cuadro 4 y Figura 1).

K	Inercia	Iteraciones
1	9230.56	2
2	5209.84	13
3	3058.87	13
4	2350.02	27
5	1929.28	22
6	1547.74	31
7	1322.72	49
8	1174.64	21

Cuadro 4: Inercia en el conjunto de entrenamiento para cada valor de K .

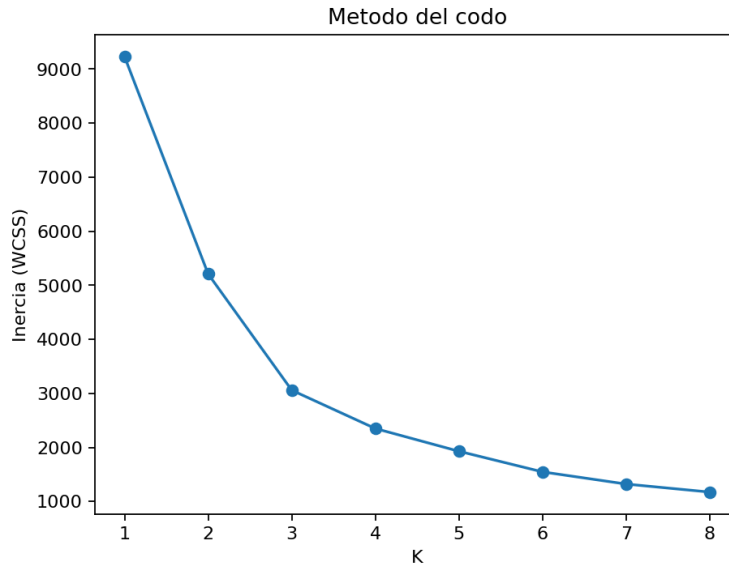


Figura 1: Método del codo: inercia contra número de clusters.

De $K = 1$ a $K = 2$ la inercia baja 4,021 y de $K = 2$ a $K = 3$ baja 2,151, pero de $K = 3$ a $K = 4$ solo baja 709 y de ahí en adelante las bajadas son cada vez más chicas. El codo está en $K = 3$, así que ese fue el valor que usé. Además con 3 clusters la interpretación es directa, como se ve en los resultados.

8. Resultados del entrenamiento

Con $K = 3$ el algoritmo convergió en 13 iteraciones, es decir, en la iteración 13 los centroides ya no cambiaron. La Figura 2 muestra cómo bajó la inercia: casi toda la mejora ocurre en las primeras 3 iteraciones y después se estabiliza.

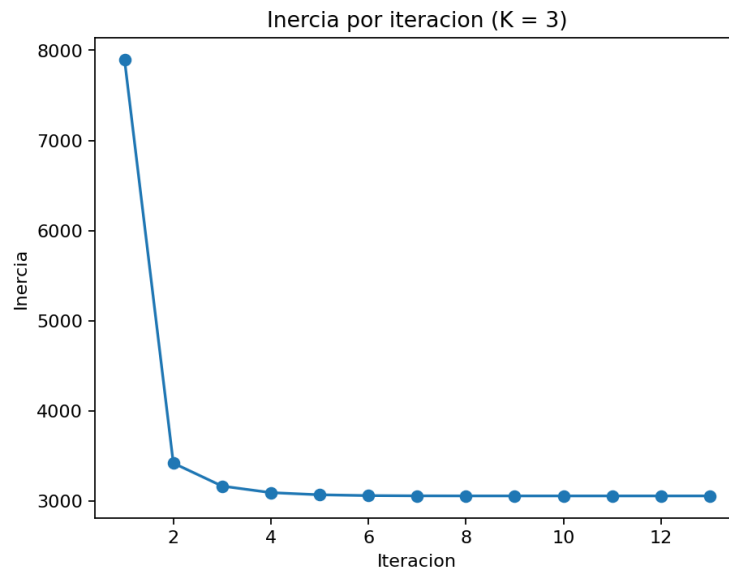


Figura 2: Inercia en cada iteración del entrenamiento con $K = 3$.

El Cuadro 5 muestra los centroides finales. La primera parte está en las unidades escaladas con las que trabaja el algoritmo; la segunda es el promedio de $\mathbf{s}$ y $\mathbf{a}$ de los registros de cada

cluster en unidades originales, que es lo que permite interpretarlos.

Cluster	Centroide escalado (s, a)	Registros	$\bar{s}$ (yd/s)	$\bar{a}$ (yd/s ²)	Interpretación
0	(−0,833, −0,688)	1951	1.19	1.12	Detenido o caminando
1	(1,172, −0,208)	1327	5.70	1.80	Corriendo rápido y estable
2	(0,029, 1,247)	1294	3.13	3.87	Acelerando

Cuadro 5: Centroides finales y descripción de cada cluster en el conjunto de entrenamiento.

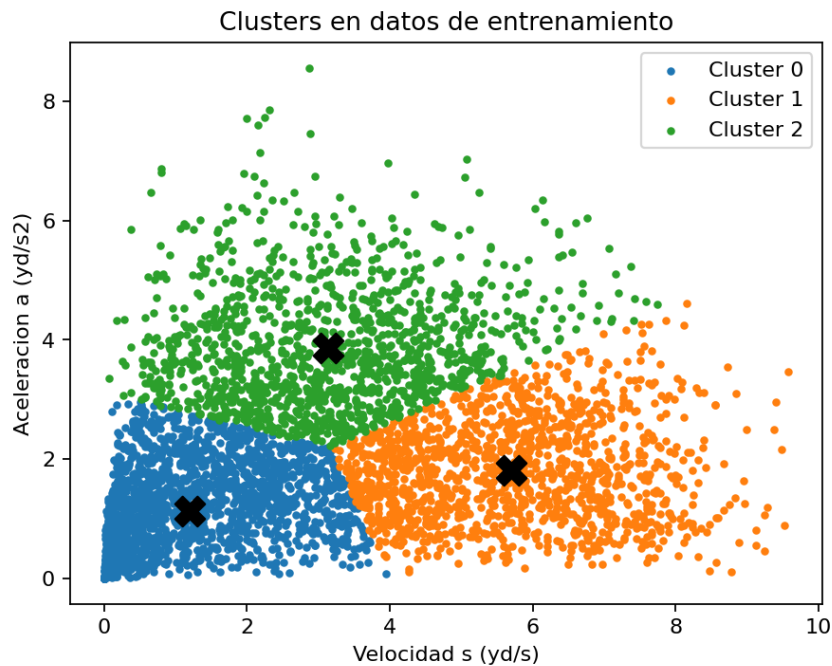


Figura 3: Clusters en el conjunto de entrenamiento. Cada color es un cluster y la X negra es su centroide.

En la Figura 3 se ve que los tres grupos quedan en zonas claras del plano velocidad-aceleración: el cluster 0 en la esquina de valores bajos, el cluster 1 a la derecha (velocidad alta, aceleración baja o media) y el cluster 2 arriba (aceleración alta).

9. Asignación de los datos de prueba

Para comprobar que el algoritmo puede recibir datos nuevos, tomé los 1,143 registros de prueba y los asigné a un cluster con mi función `asignar_clusters`, usando los centroides que salieron del entrenamiento (los centroides no se recalculan en esta etapa). El Cuadro 6 muestra los primeros 10 registros de prueba con el cluster que les tocó.

s (yd/s)	a (yd/s ²)	Rol del jugador	Cluster asignado
0.65	2.60	Defensive Coverage	0
3.55	0.37	Defensive Coverage	0
6.78	3.64	Other Route Runner	1
2.45	2.33	Passer	0
1.78	3.10	Defensive Coverage	2
0.18	1.80	Passer	0
1.45	1.53	Defensive Coverage	0
6.20	2.22	Defensive Coverage	1
1.75	1.49	Defensive Coverage	0
0.89	2.13	Defensive Coverage	0

Cuadro 6: Ejemplos de registros de prueba y el cluster asignado por la implementación manual.

Las asignaciones tienen sentido: los registros con velocidad alta (6.78 y 6.20) quedan en el cluster 1, el registro con velocidad media y aceleración alta (1.78, 3.10) queda en el cluster 2, y los que están casi detenidos quedan en el cluster 0. El Cuadro 7 compara cómo se reparten los registros entre clusters en entrenamiento y en prueba, y la Figura 4 muestra los datos de prueba con su cluster.

Cluster	Entrenamiento	%	Prueba	%
0	1951	42.7	473	41.4
1	1327	29.0	338	29.6
2	1294	28.3	332	29.0
Total	4572	100	1143	100

Cuadro 7: Registros por cluster en entrenamiento y en prueba.

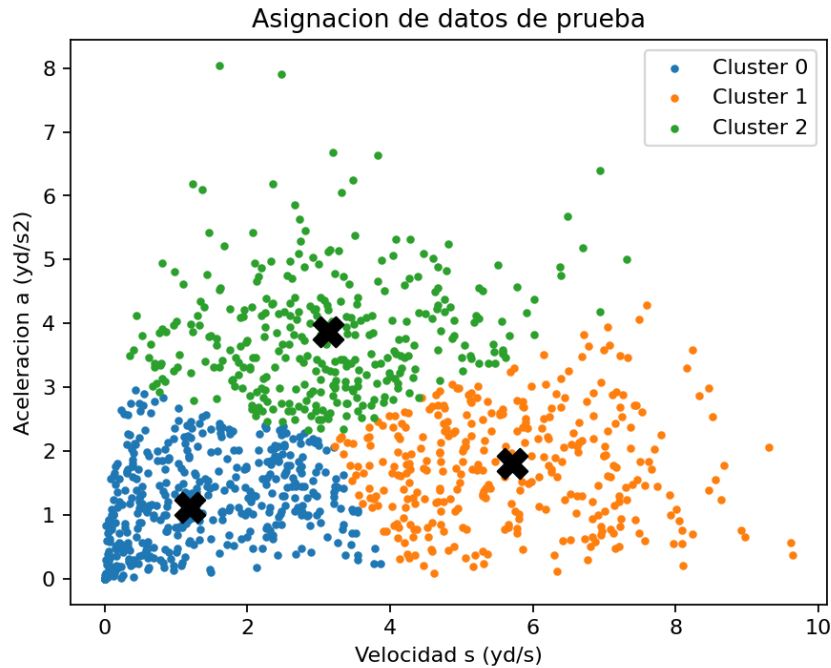


Figura 4: Datos de prueba asignados a los clusters. Las X negras son los centroides obtenidos en el entrenamiento.

10. Métricas y matriz de confusión

10.1. Métricas seleccionadas

Como K-Means es clustering, las métricas que usé son las que vimos en clase para evaluar clusters:

- **Inercia** (distancia intracluster): suma de las distancias al cuadrado de cada punto a su centroide. Entre más chica, más compactos son los clusters. Para poder comparar entrenamiento y prueba, que tienen distinto número de registros, también la divido entre el número de puntos.
- **Índice de Dunn** (distancia intercluster): mínima distancia entre clusters dividida entre la máxima distancia dentro de un cluster. Entre más grande, más separados están los clusters. Lo calculé tomando la distancia entre centroides como distancia entre clusters y el promedio de la distancia de los puntos a su centroide como distancia dentro del cluster.

Las dos las calculé con mis propias funciones, usando los mismos centroides del entrenamiento (Cuadro 8).

Métrica	Entrenamiento	Prueba
Registros	4572	1143
Inercia total	3058.87	753.81
Inercia promedio por punto	0.6690	0.6595
Índice de Dunn	2.1979	2.3364

Cuadro 8: Métricas de evaluación en entrenamiento y en prueba, calculadas con la implementación manual.

La inercia promedio por punto en prueba (0.6595) es prácticamente igual a la de entrenamiento (0.6690), y el índice de Dunn en prueba (2.34) es incluso un poco mayor. Esto quiere decir que los datos nuevos quedan tan cerca de los centroides como los datos con los que se entrenó, o sea que los centroides no están ajustados solo a la muestra de entrenamiento sino que representan bien los tipos de movimiento en general.

10.2. Por qué no incluyo una matriz de confusión

La matriz de confusión compara la clase real de cada dato con la clase que predijo el modelo, y para eso se necesita una etiqueta verdadera. En este proyecto no existe esa etiqueta: no hay una columna que diga si un registro es “detenido”, “acelerando” o “corriendo”; esos grupos los descubrió el algoritmo. Las únicas categorías del dataset (`player_role`, `player_position`) no son los clusters que estoy buscando: un mismo receptor puede estar detenido en un frame y a máxima velocidad en otro, así que compararlas contra los clusters no diría nada sobre si K-Means funciona bien o mal. Forzar una matriz de confusión mapeando clusters a roles sería inventar una evaluación que no mide lo que el algoritmo hace. Por eso la evaluación se basa en las métricas de clustering del apartado anterior, que sí miden lo que K-Means trata de lograr: clusters compactos y separados.

11. Análisis de resultados

Los resultados muestran que la implementación manual hace lo que se espera de K-Means:

- La inercia baja en cada iteración y el algoritmo se detiene solo cuando los centroides dejan de cambiar (13 iteraciones para $K = 3$), que es el criterio de paro que vimos en clase.

- El método del codo, calculado con mi propia inercia, da una curva con la forma esperada y un codo claro en $K = 3$.
- Los tres clusters se pueden interpretar sin forzar nada: jugadores casi detenidos, jugadores acelerando y jugadores corriendo a velocidad alta y estable. Es justo lo que uno esperaría de datos de movimiento en una jugada de fútbol americano, donde los jugadores arrancan, aceleran y luego mantienen la velocidad.
- Los datos de prueba se reparten entre los clusters casi en la misma proporción que los de entrenamiento (41 %, 30 %, 29 % contra 43 %, 29 %, 28 %), y sus métricas son prácticamente iguales, así que la asignación de datos nuevos funciona bien.

También hay cosas que hay que tener en cuenta. Los clusters no tienen fronteras naturales muy marcadas: la nube de puntos es continua y K-Means simplemente la parte en tres zonas, que es una de las limitaciones del algoritmo que vimos (asume clusters de forma más o menos esférica). El resultado depende de los centroides iniciales; en mi caso son fijos gracias al `random_state`, pero con otros puntos iniciales podría converger a una partición un poco distinta. Y como trabajé con una muestra de 1 de cada 50 registros, los números exactos cambiarían con otra muestra, aunque la forma de los clusters debería ser la misma porque la muestra cubre todos los partidos del archivo.

12. Conclusión

Implementé K-Means completamente a mano, con funciones sencillas para la distancia euclidiana, la elección de centroides iniciales, la asignación de puntos, la actualización de centroides, las iteraciones y el criterio de paro, además de la asignación de datos nuevos. Todo lo que no es K-Means (leer el dataset, muestrear, estandarizar, separar entrenamiento y prueba y graficar) lo hice con las herramientas que usamos en clase.

Entrené con 4,572 registros y probé con 1,143 registros que el algoritmo no había visto. Con $K = 3$, elegido con el método del codo, obtuve tres tipos de movimiento fáciles de interpretar, y las métricas en prueba (inercia promedio por punto de 0.66 e índice de Dunn de 2.34) son prácticamente iguales a las de entrenamiento, lo que demuestra que los centroides aprendidos sirven para agrupar datos nuevos. Con esto queda demostrado que el algoritmo está bien implementado y funciona para el problema planteado.